

CONDOMINIUM LOTTERY REGISTRATION SYSTEM

A Java OOP Implementation

Submitted To:	Abdisa K.
Language:	Java SE (JDK 8+)
Architecture:	Layered (Presentation → Service → Model → Persistence)
Submission:	Last week of second semester
Date:	June 4, 2026

Abstraction

Encapsulation

Inheritance

Polymorphism

Interfaces

Generics

Exception
Handling

SECTION 1

Project Title & Description

The **Condominium Lottery Registration System** is a console-based Java application designed to facilitate fair, randomized allocation of condominium unit type options to applicants. The system provides streamlined management of applicants, registrations, and lottery draws.

Business Context

- Register applicants who wish to participate in unit allocation
- Create registrations linking applicants to desired unit types
- Approve eligible registrations for lottery participation
- Conduct fair, randomized draws to select winners
- Track results and announce winners

Solution Overview

This system uses a menu-driven console interface with file-based persistence (pipe-delimited text files) to implement the complete lottery workflow. All data is automatically saved after each operation, ensuring data consistency and preventing loss due to unexpected termination.

SECTION 2

System Architecture

The system follows a **Layered Architecture** with clear separation of concerns, achieving loose coupling between the presentation, business logic, and data layers.

Layer	Component(s)	Responsibility
Presentation	Main.java	Console UI, menu flow, user I/O, lottery trigger
Service / Business	ApplicantService RegistrationService	CRUD logic, validation, auto-save
Model / Entity	Person, Applicant, Admin Registration	Data structures & accessors
Utilities	Enums, Exceptions	Types, statuses, error handling
Persistence	data/*.txt	Pipe-delimited text file storage

SECTION 3

Classes & Their Roles

BASE CLASS

- **Person** — Base class defining common fields (*id*, *fullName*, *phone*) and method `displayInfo()`.

CONCRETE PERSON TYPES

- **Applicant** (extends Person) — Represents lottery registrants. Adds *email*, *address*, *registrationDate*, overrides `displayInfo()`.
- **Admin** (extends Person) — Represents system administrators. Adds *role*, *department*, overrides `displayInfo()`.

ENTITY CLASSES

- **Registration** — Links an Applicant to a desired UnitType. Lifecycle: **PENDING** → **WON / LOST**.

SERVICE INTERFACE & IMPLEMENTATIONS

- **Manageable<T>** (Generic Interface) — Defines standard CRUD operations (add, displayAll, searchById, update, delete) for all services.
- **ApplicantService** — Implements Manageable<Applicant>. Handles CRUD, validation, and auto-save.
- **RegistrationService** — Implements Manageable<Registration>. Handles CRUD, pending lists, and auto-save.

EXCEPTION TYPES

- **RegistrationException** (checked) — Thrown for business rule violations.

SECTION 4

Object-Oriented Programming Concepts Applied

This project successfully integrates over **95%** of the core and advanced OOP concepts as required by the academic guidelines.

A. ABSTRACTION

Implementation details are hidden, exposing only essential features.

- The `Manageable<T>` interface abstracts CRUD operations, providing a consistent contract without dictating implementation details.
- The Service Layer hides complex file I/O from the Presentation layer.

B. ENCAPSULATION

Data and methods operating on that data are bundled together.

- All model fields are `private` and accessed through `public` getters/setters.
- Services encapsulate in-memory lists, hiding them from the Main class.
- Validation happens internally, ensuring data integrity.

C. INHERITANCE

Code reuse achieved by allowing subclasses to inherit from a parent class.

- `Applicant` and `Admin` extend the `Person` class.
- They inherit common fields and eliminate redundancy while adding specific fields.
- Constructor chaining (`super()`) initializes parent fields.

D. POLYMORPHISM & DYNAMIC BINDING

The same method call produces different behavior based on runtime object types.

- `Applicant` and `Admin` override the `Person.displayInfo()` method.
- The System Directory demonstrates dynamic binding using a single `List<Person>` to store mixed subtypes.

E. INTERFACES & GENERICS

Providing type-safe templates and behavior contracts.

- `Manageable<T>` ensures type-safe CRUD implementations, eliminating the need for casting.
- Interfaces enforce a consistent structure, making the system highly extensible.

F. ENUMERATIONS (ENUM)

Providing a type-safe set of named constants.

- `UnitType` — `STUDIO`, `ONE_BEDROOM`, `TWO_BEDROOM`, `THREE_BEDROOM`.
- `RegistrationStatus` — `PENDING`, `APPROVED`, `WON`, `LOST`.

G. EXCEPTION HANDLING

Separating normal execution flow from error cases gracefully.

- Standard `try-catch` blocks used comprehensively for error control throughout the application.
- `RegistrationException` (checked) manages strict business rule violations.

H. DATA AGGREGATION (CREATIVITY)

System Statistics Dashboard adds an analytical layer beyond standard CRUD.

- Iteratively analyzes all registrations to aggregate metrics.
- Calculates total counts, filters winners, and finds the most popular `UnitType`.

SECTION 5

Data Persistence & File Handling

The system employs file-based persistence using **pipe-delimited text files**, ensuring durable storage without requiring a database engine.

- Data is saved to the `data/` directory (`applicants.txt`, `registrations.txt`).
- The `loadFromFile()` method parses pipe-delimited records into active objects using `java.util.Scanner`.
- The `saveToFile()` method serializes objects back into strings for storage.
- **Auto-save** is triggered after every mutation (add, update, delete) to prevent data loss.

SECTION 6

Actual Application Run & Output

Below is an abbreviated transcript of the actual console output demonstrating the complete lottery workflow — adding applicants, creating registrations, conducting the draw, and viewing winners.

1. Main Menu Display

```
CONDOMINIUM LOTTERY REGISTRATION SYSTEM
Developed with Beginner Java OOP Principles

MAIN MENU
=====
1. Manage Applicants
2. Manage Registrations
3. Conduct Lottery Draw
```

```
4. View Lottery Winners
5. System Directory (View All Users)
6. System Statistics Dashboard
7. Exit
```

2. Adding Applicants

```
--- Add New Applicant ---
Applicant ID: APP001
Full Name: Abebe Kebede
Phone Number: 0911223344
Email Address: abebe@email.com
Home Address: Addis Ababa
[SUCCESS] Applicant added: Abebe Kebede

Applicant ID: APP002
Full Name: Tigist Hailu
[SUCCESS] Applicant added: Tigist Hailu
```

3. Creating Registrations

```
--- Create New Registration ---
Registration ID: REG001
Applicant ID: APP001
Select Unit Type (1-4): 1 [STUDIO]
[SUCCESS] Registration added: REG001

Registration ID: REG002
Applicant ID: APP002
Select Unit Type (1-4): 1 [STUDIO]
[SUCCESS] Registration added: REG002
```

4. Conducting the Lottery Draw

```
=====
      LOTTERY DRAW IN PROGRESS...
=====
Type: STUDIO
Participants: 2
=====

*** WINNER: Applicant ID APP001 (Registration: REG001) ***

Lottery draw completed successfully!
```

5. Viewing Lottery Winners

```
===== LOTTERY WINNERS =====
Registration ID : REG001
Applicant ID   : APP001
Unit Type      : STUDIO
Date           : Thu Jun 04 22:12:46 EAT 2026
Status         : WON
```

Thank you for using the System. Goodbye!

SECTION 7

Conclusion

The **Condominium Lottery Registration System** is a robust, well-architected Java application that successfully manages a complex business workflow from applicant registration through to winner announcement.

It serves as a comprehensive demonstration of Object-Oriented Programming, featuring a cleanly decoupled layered architecture, thorough exception handling, and practical file I/O. By successfully applying **Abstraction, Encapsulation, Inheritance, Polymorphism, Interfaces, Generics, Enumerations, and Exception Handling**, the system validates a deep understanding of core and advanced OOP principles.

- Clean layered architecture with clear separation of concerns
- Generic Manageable<T> interface for extensible, type-safe CRUD
- Polymorphic Person hierarchy with dynamic dispatch
- Pipe-delimited file persistence with auto-save on every mutation
- Comprehensive exception handling for business rule enforcement
- Statistics dashboard demonstrating advanced data aggregation